

POS System

Technical Documentation

Production-Grade Point of Sale System
Kenyan Supermarket Edition

Version 1.0
8/22/2026

Tech Stack: Node.js + NestJS + Prisma + PostgreSQL
Offline Support: Electron + SQLite
Payment Integration: M-Pesa STK Push

Table of Contents

- 1. System Overview
- 2. Architecture
- 3. Database Schema (ERD)
- 4. API Endpoints
- 5. Authentication & RBAC
- 6. Payment Integration (M-Pesa)
- 7. Offline Sync Strategy
- 8. Receipt Generation
- 9. Setup & Installation
- 10. Default Credentials

1. System Overview

This is a production-grade Point of Sale (POS) system designed for Kenyan supermarkets. It handles sales transactions, inventory management, payment processing (M-Pesa, Cash, Card), customer loyalty, and business analytics.

Key Features:

- Transactional sales with automatic stock deduction and audit trail
- M-Pesa STK Push integration with idempotent callback handling
- Offline-first architecture (SQLite local + PostgreSQL cloud)
- Role-based access control with granular permissions
- Real-time reporting (revenue, profit, stock valuation)
- PDF receipt generation
- Purchase order management
- Refund workflow with segregation of duties
- Cash session management with reconciliation

2. Architecture

The system follows a modular monolith architecture with clean module boundaries.

Module Structure:

```
src/  
modules/  
  auth/      - Login, JWT, Guards, Permissions  
  products/   - Products, Variants, Categories, Brands  
  sales/      - Sales, Refunds, Receipts  
  payments/   - Payment processing, M-Pesa integration  
  customers/  - Customer management, Loyalty  
  inventory/  - Stock levels, Adjustments, Transfers  
  cash-management/- Cash sessions, Cash-in/out, Expenses  
  suppliers/  - Suppliers, Purchase Orders  
  reporting/  - Analytics, Reports  
  audit/      - Audit logging  
common/  
  decorators/ - Custom decorators (CurrentUser, RequirePermissions)  
  filters/    - Global exception filter  
  interceptors/ - Audit interceptor  
  utils/      - Receipt PDF generation
```

3. Database Schema (ERD)

Three data layers:

MASTER DATA:

- Branches, Users, Roles, Permissions
- Products, Variants, Categories, Brands
- Customers, Suppliers, Terminals, Cash Registers

TRANSACTIONAL DATA:

- Sales, Sale Items
- Payments, Payment Methods
- Refunds, Refund Items
- Stock Movements
- Cash Sessions, Cash Movements
- Purchase Orders, PO Items
- Expenses
- Audit Logs

DERIVED DATA (Views/Analytics):

- Daily Sales Summary
- Inventory Valuation
- Top Products
- Cash Reconciliation

Key Relationships:

branches -> users, terminals, sales, cash_registers, expenses, purchase_orders

products -> variants -> sale_items, stock_movements, po_items

sales -> sale_items, payments, refunds

cash_sessions -> sales, cash_movements

4. API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/login	Login with username + PIN	No
POST	/api/auth/refresh	Refresh JWT token	Bearer
GET	/api/products	List products (search, filter)	Bearer
GET	/api/products/scan/:barcode	Barcode lookup	Bearer
POST	/api/products	Create product	Bearer
PUT	/api/products/:id	Update product	Bearer
POST	/api/products/:id/variants	Add variant	Bearer
POST	/api/sales	Create sale	Bearer
GET	/api/sales	List sales	Bearer
GET	/api/sales/:id	Sale detail	Bearer
GET	/api/sales/:id/receipt	Generate PDF receipt	Bearer
GET	/api/sales/daily-summary	Daily sales summary	Bearer
POST	/api/sales/refunds	Create refund	Bearer
GET	/api/sales/refunds	List refunds	Bearer
POST	/api/sales/refunds/:id/approve	Approve refund	Bearer
POST	/api/sales/refunds/:id/reject	Reject refund	Bearer
POST	/api/payments/initiate	Initiate payment	Bearer
POST	/api/payments/callback/mpesa	M-Pesa callback	No
GET	/api/payments/:id/status	Check payment status	Bearer
GET	/api/inventory	Stock levels	Bearer
GET	/api/inventory/low-stock	Low stock items	Bearer
GET	/api/inventory/:variantId/movements	Stock movement history	Bearer
POST	/api/inventory/adjust	Manual stock adjustment	Bearer
POST	/api/inventory/transfer	Transfer between branches	Bearer
POST	/api/inventory/damage	Record damaged stock	Bearer
GET	/api/customers	List customers	Bearer
GET	/api/customers/phone/:phone	Find by phone	Bearer
POST	/api/customers	Create/upsert customer	Bearer
POST	/api/cash-sessions/open	Open cash session	Bearer
POST	/api/cash-sessions/:id/close	Close cash session	Bearer
GET	/api/cash-sessions/:id/summary	Session summary	Bearer
POST	/api/cash-sessions/:id/cash-in	Cash in	Bearer
POST	/api/cash-sessions/:id/cash-out	Cash out	Bearer
POST	/api/cash-sessions/:id/petty-cash	Petty cash	Bearer
GET	/api/suppliers	List suppliers	Bearer
POST	/api/suppliers	Create supplier	Bearer
POST	/api/purchase-orders	Create purchase order	Bearer
GET	/api/purchase-orders	List purchase orders	Bearer
GET	/api/purchase-orders/:id	PO detail	Bearer
POST	/api/purchase-orders/:id/receive	Receive stock	Bearer
POST	/api/purchase-orders/:id/cancel	Cancel PO	Bearer
GET	/api/reports/daily-summary	Daily summary report	Bearer
GET	/api/reports/top-products	Top products report	Bearer
GET	/api/reports/cash-reconciliation	Cash reconciliation	Bearer
POST	/api/expenses	Record expense	Bearer
GET	/api/expenses	List expenses	Bearer
GET	/api/expenses/summary	Expense summary	Bearer

5. Authentication & RBAC

Authentication:

- JWT-based authentication with 8-hour token expiry
- Login with username + 4-digit PIN
- Bearer token in Authorization header

Roles:

- super_admin: Full system access
- branch_manager: Branch-scoped, reports, refunds, inventory
- cashier: POS operations only
- inventory_clerk: Stock management only
- accountant: Reports and financial data only

Permissions (examples):

- sale.create, sale.view, sale.refund.create, sale.refund.approve
- payment.initiate, payment.callback.receive
- inventory.view, inventory.adjust, inventory.transfer
- product.create, product.update, product.delete, product.view
- customer.create, customer.update, customer.view
- report.view_sales, report.view_profit, report.view_inventory
- cash.session.open, cash.session.close, cash.session.view

6. Payment Integration (M-Pesa)

Payment State Machine:

PENDING -> PROCESSING -> SUCCESS/FAILED/TIMEOUT

Flow:

1. Cashier initiates payment with sale ID and phone number
2. System creates Payment record (status: PENDING)
3. STK push sent to customer phone
4. Payment status updated to PROCESSING
5. Customer enters PIN on phone
6. Safaricom sends callback to /api/payments/callback/mpesa
7. System verifies callback and updates payment status
8. If successful, sale payment status updated

Idempotency:

- Every payment has a unique idempotency_key
- Duplicate callbacks are detected and rejected
- Receipt numbers (MpesaReceiptNumber) used as deduplication key

Configuration (.env):

- MPESA_CONSUMER_KEY, MPESA_CONSUMER_SECRET
- MPESA_PASSKEY, MPESA_SHORTCODE
- MPESA_CALLBACK_URL

7. Offline Sync Strategy

Architecture:

- Electron app with local SQLite database
- Sales work offline, queue for sync
- Master data (products, prices) synced from cloud

Sync Flow:

1. POS creates sale in local SQLite
2. Sale added to sync_queue table
3. Background sync engine pushes queued sales
4. Server assigns sale_number and validates
5. Master data pulled down on reconnect

Conflict Resolution:

- Sales: Last-write-wins with server timestamp
- Stock: Server is source of truth
- Master Data: Server overwrites local changes
- Idempotency keys prevent duplicate transactions

Sync Queue Table (SQLite):

id, operation, table_name, record_id, payload, created_at, synced, retry_count

8. Receipt Generation

PDF receipts are generated using pdfkit with thermal receipt format (80mm).

Receipt Contents:

- Branch name and header
- Sale number, date, time
- Cashier name
- Item list with quantities, prices, discounts
- Subtotal, VAT (16%), Total
- Payment methods and references
- Loyalty points earned
- Thank you message (English + Swahili)

Endpoint: GET /api/sales/:id/receipt

Returns: PDF file with Content-Disposition header

9. Setup & Installation

Prerequisites:

- Node.js 18+
- PostgreSQL 14+
- npm or yarn

Installation:

1. `cd D:\pos-backend`
2. `npm install`
3. Configure `.env` (database URL, JWT secret, M-Pesa keys)
4. `npx prisma migrate dev --name init`
5. `npx prisma db seed`
6. `npm run start:dev`

Available Scripts:

- `npm run start:dev` - Start in development mode
- `npm run build` - Build for production
- `npm run start:prod` - Start production server
- `npx prisma studio` - Open Prisma Studio
- `npx prisma db seed` - Seed database

Swagger Documentation:

Available at <http://localhost:3000/docs>

10. Default Credentials

Admin User:

Username: admin

PIN: 1234

Cashier User:

Username: cashier1

PIN: 1234

Note: Change these credentials in production!

--- End of Documentation ---